AI FAQ & Customer Support Automation

SYSTEM ARCHITECTURE & PROJECT DESIGN SPECIFICATION

1. ARCHITECTURE

Client → Routes → Controller → Model / Service → MongoDB / Gemini AI → JSON Response

MVC pattern cleanly separates API routing, request controllers and business logic, data models, and the dedicated Gemini AI integration service for scalable support processing.

2. Layer Design

- **Client:** Web Chatbot widget, Customer Portal, Agent Dashboard, Mobile App.
- **Middleware:** JWT Auth, Role-Based Access (Customer/Agent/Admin), Rate Limiter, CORS.
- **Routes:** Auth, FAQ/Knowledge Base, Support Tickets, AI Engine.
- **Controllers:** User Auth, Ticket lifecycle, FAQ management, AI triage logic.
- **Model/Service:** Mongoose models and Gemini AI RAG service.

3. Database Design

Collections: `Users`, `KnowledgeBase`, and `Tickets`.

- `Ticket.user` references `User._id` for customer lookup.
- `Ticket.assignedAgent` references `User._id` for assigned agents.
- Supports one-to-many relationship mapping across customer ticket histories.

4. API DESIGN

Module	HTTP Method & Endpoint	Description
Auth	`POST /api/auth/register` | `GET /profile`	User registration & authenticated profile retrieval
FAQ / KB	`GET /api/faqs` | `POST /api/faqs`	Manage and query structured knowledge base articles
Tickets	`POST /api/tickets` | `PUT /api/tickets/:id`	Submit, manage status, and escalate support tickets
AI Engine	`POST /api/ai/faq-query`	RAG-based response generation from user inquiries
AI Engine	`POST /api/ai/ticket-auto-reply`	Automated triage, sentiment tags, and auto-reply drafting

5. AI SYSTEM DESIGN